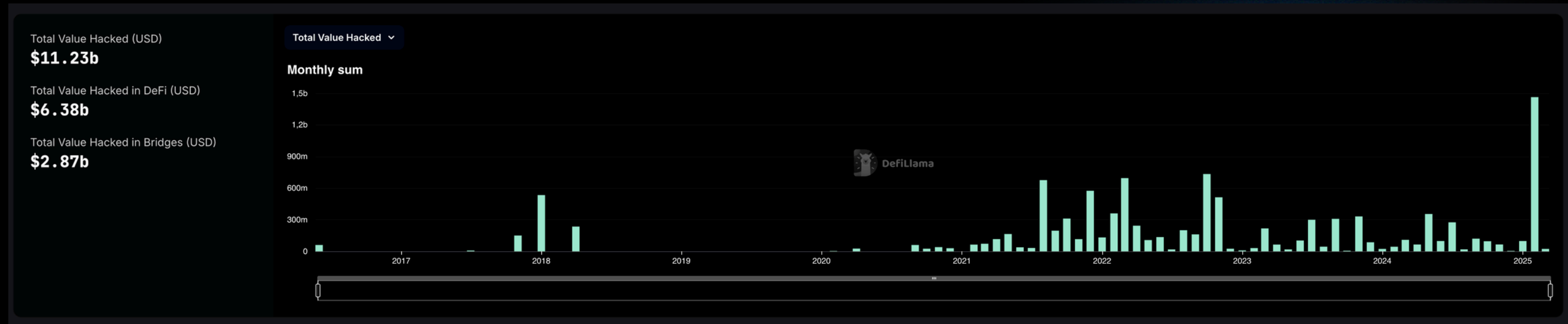


BERTolina

Large Language Model for Smart Contract
Vulnerability Detection

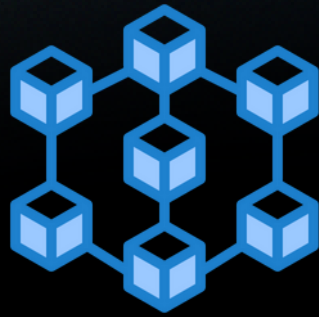
by Mattia Pietromartire

THE PROBLEM OF VULNERABILITIES IN SMART CONTRACTS



- Smart Contracts manage billions of dollars in digital assets
- Critical vulnerabilities caused losses for over \$11.23 billions from 2016
- The immutable code makes post-deployment fixes impossible
- Idea: develop an automated system to detect vulnerabilities before deployment

BACKGROUND: BLOCKCHAIN



Blockchain

- Distributed, decentralized and immutable digital ledger
- Information grouped in blocks linked to each other



Ethereum

- Programmable blockchain platform
- Decentralized applications
- Solidity



Smart Contract

- Auto-executable programs
- Immutable
- Transparent
- No intermediaries

BACKGROUND: AI

Machine Learning

- Algorithm that “teaches” computers to learn from data
- Making decisions without being programmed to do so
- Automatic experience-based machine learning



Large Language Models

- Specific type of Machine Learning, specialized in human language
- Trained on massive amounts of text to understand, generate and summarize natural language
- Powerful models for text processing and generation



SMART CONTRACT VALIDATION AND VERIFICATION: STATE OF THE ART

STATIC ANALYSIS

Examines the source code of the smart contract without executing it

Pros:

- Analyses the whole code
- Fast
- Search for known patterns

Cons:

- High number of false positives
- Difficulties with complex logic
- Limited by known patterns

DYNAMIC ANALYSIS

Examines the smart contract while is executing in a simulated environment

Pros:

- Less false positives
- Evaluates the actual behavior
- Identifies complex vulnerabilities

Cons:

- Limited covering
- Complex setup
- Slow, Time consuming

BERTOLINA: A LLM BASED APPROACH

- CodeBERT, pre-trained architecture on code and text
- Multi-label vulnerabilities classification
- Supervisioned Fine-Tuning (SFT)

Innovation: Understanding of code and semantic context

DATASET

Before

[illegible]

- Slither Audited Smart Contracts, created by Martina Rossini
- Classification with Slither labels
- Pre-processing of the dataset

After

source_code string	slither list
pragma solidity 0.5.4; interface IERC20 { function balanceOf(address account) external view returns (uint256); function transfer(address recipient, uint256 amount) external returns (bool); function...	[4]
pragma solidity 0.6.10; pragma experimental ABIEncoderV2; library BokkyPooBahsDateTimeLibrary { uint constant SECONDS_PER_DAY = 24 * 60 * 60; uint constant SECONDS_PER_HOUR = 60 * 60; uint constant...	[6,4,9,7]
pragma solidity 0.6.5; pragma experimental ABIEncoderV2; struct UserInfo { uint256 amount; uint256 rewardDebt; } struct PoolInfo { address lpToken; uint256 allocPoint; uint256 lastRewardBlock; uint256...	[38]
pragma solidity 0.6.6; library Address { /** * @dev Returns true if `account` is a contract. * * [IMPORTANT] * ==== * It is unsafe to assume that an address for which this function returns * false is a...	[38]
pragma solidity 0.6.8; pragma experimental ABIEncoderV2; contract Initializable { /** * @dev Indicates that the contract has been initialized. */ bool private initialized; /** * @dev Indicates that the...	[0,4,7,9,16,5,2]
pragma solidity 0.4.26; interface IERC20 { function totalSupply() external view returns (uint256); function balanceOf(address who) external view returns (uint256); function allowance(address owner,...	[2]

EVALUATION METRICS

Accuracy:

- % of correct predictions on the total
- General indicator of the system efficiency

F1-Score:

- Harmonic mean between precision and recall
- Indicates the effectiveness of correctly classifying vulnerabilities

Precision:

- % of true positives between cases identified as positive
- Indicates the quality of the forecasts. Reduces the efforts of manual revision

Recall:

- % of true positives actually identified
- Critical to not lose true vulnerabilities

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.3755	0.337933	84.40%	66.07%	73.52%	61.48%
2	0.3339	0.314019	86.12%	69.80%	78.97%	64.30%
3	0.2332	0.307004	87.02%	72.51%	78.24%	68.39%
4	0.2735	0.301931	87.57%	73.44%	80.92%	68.56%
5	0.2251	0.289463	88.25%	75.14%	81.49%	70.17%
6	0.1899	0.291072	88.71%	76.30%	82.44%	71.42%
7	0.2155	0.306638	88.77%	76.48%	82.30%	71.72%
8	0.1528	0.314127	89.04%	77.05%	82.71%	72.30%
9	0.1534	0.319319	89.05%	77.05%	83.19%	72.10%
10	0.1728	0.316908	89.31%	77.62%	83.40%	72.81%
11	0.1597	0.334423	89.27%	77.71%	83.01%	73.16%
12	0.163	0.340034	89.53%	78.14%	83.73%	73.42%
13	0.1314	0.352396	89.41%	77.76%	83.81%	72.91%
14	0.1195	0.356023	89.55%	78.19%	83.77%	73.49%
15	0.1058	0.368381	89.55%	78.32%	83.89%	73.60%
16	0.1306	0.376553	89.60%	78.28%	83.67%	73.69%
17	0.1221	0.388485	89.65%	78.52%	83.77%	73.97%
18	0.1184	0.394911	89.71%	78.54%	84.16%	73.80%
19	0.1253	0.402235	89.79%	78.68%	84.17%	73.99%
20	0.1006	0.411493	89.97%	79.03%	84.71%	74.27%
21	0.1058	0.417936	89.89%	78.89%	84.56%	74.08%
22	0.1079	0.420763	89.88%	78.87%	84.45%	74.15%

CONFRONTO

Model	Dataset	Accuracy	Precision	Recall	F1-Score	Note
BERTolina	106.474 smart contract (15% test, 15% validation)	89,97%	84,71%	74,27%	79,03%	Source code analysis
Eth2Vec	95.152 smart contract	–	77%	50,7%	57,5%	Opcode analysis
VulnHunt-GPT	69 smart contract	–	–	–	–	Metrics not provided

CONCLUSIONS

Main contributions:

- Working model for automatic and multi-label vulnerability detection
- Excellent precision and recall results compared to similar products
- Falsi positives cut

Future developments:

- Optimization of the code
- Implementation of opcode analysis
- Extension to emerging languages (Vyper, Rust)